Plan de acción · III Malackathon

E.T.S.I. Informática · Universidad de Málaga · 15 y 16 de octubre de 2026

Documento preparado el 13/10/2026 (actualizado el 14/10 por la tarde) · Equipo de Javier Román Meléndez · Carpeta del proyecto: `C:\Users\jrmro\Malackathon_2026` · Repo privado: github.com/javirmmn/malackathon-2026

Lo crítico en una línea: los 4 ya tenemos cuenta de Oracle (@uma.es, de una asignatura), así que **no hace falta la encuesta**. Cada uno debe comprobar **hoy** que puede entrar con el doble factor. El jueves 15 hay que **entregar por el Campus Virtual antes de las 20:00**. Si no hay entrega, el equipo no cuenta.

Índice

1. Resumen del evento
2. Programa y normas
3. Retos y nuestro orden de preferencia
4. Infraestructura Oracle (OCI)
5. Checklist: hoy, 13/10
6. Mañana, 14/10: kits de arranque (NICS y Mercedes)
7. Organización del equipo
8. Plan horario del jueves 15
9. Presentación y defensa (viernes 16)
10. Plan por reto: NICS Lab
11. Plan por reto: Mercedes-Benz
12. Plan por reto: NTT DATA
13. Plan por reto: Indra
14. Plan por reto: Google
15. Plan por reto: Betsson
16. Material e instalación previa
17. Riesgos y planes de contingencia
18. Contactos y enlaces
19. Contexto para retomar con Claude

1. Resumen del evento

- **Qué es:** 3.ª edición del Malackathon, el hackathon de la ETSI Informática de la UMA. Lo organiza la Escuela con Betsson, Grupo Indra, Google, Mercedes-Benz, NTT DATA y NICS Lab UMA. Oracle es colaborador tecnológico y la Cátedra OPPLUS patrocina.
- **Formato:** equipos de 4 estudiantes de grado de la ETSII. Cada empresa propone su propio reto y lo resuelven como máximo unos **15 equipos por reto**. Toda la documentación y la comunicación son en castellano.
- **Asignación de retos:** según la preferencia del equipo y el orden de envío del cuestionario (el primero en llegar tiene prioridad). La organización publica en el Campus Virtual el listado de equipos por empresa y reto.
- **Premios:** cada empresa premia al equipo que considera ganador de su reto. Hay varios premios por categoría y reto, todavía sin anunciar.
- **Restricción:** no se admiten miembros con experiencia profesional significativa.
- **Lugar de trabajo:** cada equipo tiene un aula asignada (habrá un PDF de ubicaciones en el Campus Virtual) y debe trabajar desde su puesto toda la jornada.
- **Evaluación:** durante el día 1, la organización y los mentores de cada empresa van evaluando las distintas fases del trabajo. Con esas evaluaciones se eligen los finalistas.

2. Programa y normas

Cuándo	Qué
Jue 15 · 09:00	Registro en la antesala del Salón de Actos
09:30	Presentación general del hackathon (Salón de Actos)
09:45	Presentación de cada reto en su aula. El aula de cada reto se publica en el Campus Virtual
10:15 – 20:00	Trabajo en las aulas, con mentores de la empresa, profesores de la Escuela y estudiantes de máster
antes de 20:00	**Entrega por el Campus Virtual o el formulario habilitado**. Solo se considera presentado el equipo que entrega
21:45	Se publica la lista de finalistas en el Campus Virtual
Vie 16 · 09:30	Recepción de finalistas (Salón de Actos)
10:00	Presentaciones de los finalistas: **7 minutos como máximo**, demo en directo
12:30	Deliberación del jurado
13:00	Entrega de premios

NORMAS CLAVE DE LA PRESENTACIÓN

- Presenta **un único miembro** del equipo, con una **demostración en directo**.
- Se puede proyectar un **PDF de apoyo** y enseñar cosas en vivo.
- Después de todas las presentaciones hay un debate con el jurado en el que puede hablar cualquier miembro. El jurado puede pedir que se muestren la **funcionalidad y la arquitectura**.
- Las decisiones del jurado son inapelables.
- Si el equipo abandona, hay que avisar en la sala de co-working **Haz Cosas (aula 4.1.1)**, que es la sede de la organización durante el evento.

3. Retos y nuestro orden de preferencia

Orden enviado: **1 NICS Lab · 2 Mercedes-Benz · 3 NTT DATA · 4 Indra · 5 Google · 6 Betsson**. **El reto asignado no se conoce hasta el jueves 15 por la mañana.** Por eso el 14/10 se preparan kits de arranque para las dos primeras opciones (apartado 6).

#	Empresa	Reto	Resumen	Dificultad
1	NICS Lab	Infiltrados en la red: encuentra a la IA impostora	Una red corporativa expone decenas de servicios web. Hay que descubrirlos y clasificar cada uno como *real*, *honeypot clásico* u *honeypot generativo* (respuestas inventadas por una IA), documentando qué delató a cada impostor.	2–3 la básica; 4 la de IA
2	Mercedes-Benz	Clustering the Star	Aprendizaje no supervisado para segmentar la base de clientes y sacar estrategias de marketing personalizadas e insights de negocio.	3

3	NTT DATA	El Guardián: detecta cuándo una IA se inventa las cosas	Detector de alucinaciones: lee la respuesta de una IA junto con sus fuentes y decide si se ha inventado algo. LLMs locales, texto en español, ranking en directo y una prueba sorpresa a mitad de evento. Dan un bot de ejemplo que ya funciona.	4/5
4	Grupo Indra	Detección y caracterización de un incendio por satélite	Dos imágenes Sentinel-2 (Copernicus) de la misma zona e instante, de un incendio en la Comunidad de Madrid: una en color real (True Color) y otra con infrarrojo de onda corta (SWIR).	4
5	Google	Incidente en el Google Safety Engineering Center	Hacer de SOC de emergencia: desplegar un repositorio de datos propio y una consola web para analizar 30 días de telemetría sintética, reconstruir la intrusión y contener la amenaza en vivo.	3 básico; 4 avanzado
6	Betsson	Casino Lobby, lanzamiento y jugada	Aplicación que compruebe por sí sola que un juego nuevo aparece en el lobby, lo lance y juegue una partida en webs reales de Betsson, dejando capturas y un veredicto.	4–5

4. Infraestructura Oracle (OCI)

4.1 Situación actual (actualizada el 14/10)

- **Los 4 miembros tienen cuenta OCI** de una asignatura anterior, asociada a su @uma.es. La principal es la de Javier: tenancy `javierroman`, región de origen **Spain Central (Madrid)**, en estado *Free Trial*.
- **Correo de la organización (13/10):** la encuesta es solo para quien **todavía no tiene cuenta**. Hay que ser mayor de edad, usar un @uma.es y **no se puede repetir si ya se pidió**. La alternativa es un alias @uma.es.
- **Decisión: no rellenamos la encuesta**. Trabajamos con la tenancy de Javier y las de los demás quedan de respaldo.
- **Base de datos antigua `SeguridadOracle`: ya no existe.** No aparece en *Autonomous AI Databases* del compartimento raíz. Oracle la habrá eliminado por inactividad (más de 120 días sin uso). El enlace de "My recently viewed" del inicio daba 404 porque esa lista no se actualiza. **No hay nada que borrar** y los **2 huecos Always Free están libres**.
- **Base de datos del evento creada el 14/10:** Database name `malackathon` (Display name `malackathon-2026`), Serverless 26ai, Transaction Processing, **Always Free**, Madrid, acceso desde cualquier sitio con mTLS. Usuario `MALACK` creado y ensayo de carga superado. La URL del esquema y los detalles están en `docs/oracle.md`. **Queda 1 hueco Always Free libre.**

Límites que hay que conocer:

- Como máximo **2 Autonomous Databases *Always Free*** por tenancy.
- Una base de datos Always Free que pasa **7 días sin uso se detiene** (se puede volver a arrancar con *Start*), y si sigue detenida unos 90 días **se elimina**. Una base de datos creada hoy sigue activa el jueves.
- Recursos Always Free: 1 OCPU y 20 GB de almacenamiento por base de datos. Suficiente para el hackathon.
- Cuando termina el trial, la cuenta queda limitada a recursos Always Free. Por eso se crea **siempre** con el interruptor **Always Free** activado.

4.2 Encuesta de infraestructura (no aplica a nuestro equipo)

- Encuesta "Utilización de Infraestructura de Oracle" en el Campus Virtual. Plazo: **13/10 a las 23:59**.
- Se rellena **una sola vez por equipo**. Queda registrado quién la envía, así que conviene que lo haga el responsable.
- Solo vale un correo **@uma.es**. **No aplica a nuestro equipo:** los 4 ya pedimos cuenta en el pasado y no se puede repetir. Si alguna cuenta resultara inutilizable, la alternativa que da la organización es un **alias @uma.es**.

4.3 Flujo técnico según la guía "Crear Base de Datos en OCI"

1. Entrar en la consola de OCI con el doble factor ya configurado. El idioma se puede cambiar.
2. Buscar *Oracle Database* y crear una **Base de Datos Autónoma** de **Procesamiento de Transacciones** (ATP), marcando **SIEMPRE GRATIS / Always Free**. El resto de opciones, por defecto.
3. Se crea el usuario `ADMIN`. **Apuntad su contraseña: es muy importante.**
4. *Acciones de base de datos* → *Usuarios de base de datos*: crear un usuario nuevo con contraseña fuerte, cuota **UNLIMITED** y los roles **Gráfico**, **OML** y **REST** activados.
5. Copiar la **URL de acceso al esquema** de ese usuario (aparece debajo del usuario en la lista). **Se entrega después en el formulario de evaluación.**
6. Cargar los datos: abrir la URL del usuario, iniciar sesión y entrar en *Carga de datos* → *Carga de datos*. Arrastrar el Excel descargado del Campus Virtual.
7. **Muy importante:** pulsar el **lápiz** para configurar la importación. Se recomienda poner como `VARCHAR2` las fechas, países y campos problemáticos, y transformarlos después con SQL.
8. Pulsar **Inicio** para importar y después **Listo**.
9. Opcional: montar un servidor web en una instancia de Compute. No es obligatorio; está explicado en el vídeo del taller.

5. Checklist del 13/10 (Oracle completado el 14/10)

Ya hecho (13/10 por la tarde y 14/10 por la mañana):

- **Repositorio privado** en GitHub: github.com/javirmmn/malackathon-2026 (rama `main`), con la estructura del apartado 6.1, `.gitignore` que bloquea `.env`, wallet y `data/`, y un `README` de "arrancar en 5 minutos". **Los otros 3 ya son colaboradores** (14/10). Javier trabaja con GitHub Desktop sobre la carpeta local.
- **Oracle (14/10):** base de datos `malackathon` creada (Always Free, 26ai, Madrid), usuario `MALACK` con Graph/OML/REST y URL del esquema guardada en `docs/oracle.md`. **Ensayo de carga superado:** 10 filas, tildes correctas, fecha y decimal con coma convertidos con SQL y tablas de prueba borradas. La configuración de Data Load que funcionó está anotada en `docs/oracle.md`. **Pendiente:** wallet y prueba de conexión desde Python (paso 6, opcional).
- **Python** en el portátil de Javier: 3.13.7 con entorno `.venv` y todas las dependencias comunes **y las de los dos kits** (hdbscan, umap-learn, kmodes, httpx, fastapi…) instaladas y comprobadas.
- **Partes comunes del 14/10 adelantadas** (`common/`): conexión a Oracle por `.env` (`query_df`, `execute`, `df_to_table`), lectura de CSV/Excel con tildes y decimales con coma, estilo de gráficos y plantilla de dashboard Streamlit. 4 tests pasan.
- **SQL del ensayo** listo en `docs/sql/prueba_carga.sql` y plantilla `docs/oracle.md` para la URL del esquema.

5.1 Ensayo de Oracle: crear ya la base de datos del evento (Javier, unos 30 min)

`hecho el 14/10`

En lugar de una base de datos "de usar y tirar", **se crea hoy la que usaremos el jueves**. Así llegamos con todo comprobado y ahorramos tiempo el día 1. Si el reto exigiera otra configuración, queda libre un segundo hueco Always Free.

1. **Crear la base de datos.** ☰ → *Oracle Database* → *Autonomous Database*, con la región **Spain Central (Madrid)** y el compartimento `javierroman (root)` → **Create Autonomous AI Database**.
   - Display name `malackathon-2026` y Database name `malackathon`. El Database name solo admite letras y números y no se puede cambiar; el Display name, sí.
   - Workload type: **Transaction Processing**.
   - Interruptor **Always Free**: **activado**. Comprobad que no se cobra nada.
   - Contraseña de `ADMIN`: fuerte, **guardada en un gestor de contraseñas**. Nunca en el repo.
   - Acceso de red: el que venga por defecto (acceso seguro desde cualquier sitio).
   - Lo demás, por defecto → *Create*. Tarda de 3 a 5 minutos hasta quedar en estado **Available**.
2. **Crear el usuario de trabajo.** Dentro de la base de datos: *Database actions* → *Database Users* → *Create User*.
   - Usuario: `MALACK` (o el nombre del equipo), con contraseña fuerte.
   - Quota on tablespace DATA: **UNLIMITED**.
   - Activar **Graph**, **OML** y **Web Access (REST)**.
3. **Copiar la URL del esquema** que aparece en la tarjeta del usuario. Pegarla en el hueco marcado de `docs/oracle.md` (ya existe; solo la URL, sin contraseñas), hacer commit y push. **Es la que se entrega en el formulario de evaluación.**
4. **Probar la carga de datos.** Abrir la URL, iniciar sesión como `MALACK` → *Data Load* → *Load Data*. Arrastrar el fichero de prueba `prueba_carga.csv` de la carpeta del proyecto.
   - Pulsar el **lápiz** y poner como `VARCHAR2` la fecha, el país y el campo decimal con coma.

    - *Start → Done*.
5. **Comprobar con SQL.** En *SQL* (Database Actions), pegar `docs/sql/prueba_carga.sql` y ejecutarlo bloque a bloque: recuento (10 filas), tipos, tildes, conversión de la fecha con `TO_DATE(fecha,'DD/MM/YYYY')` y del decimal con coma (dos formas), detección de filas que no convierten y una tabla "limpia" tipada. Cada consulta lleva al lado el resultado esperado.
6. **Opcional, recomendable para Google o Mercedes:** descargar el **wallet** (*Database connection → Download wallet*, con contraseña), descomprimirlo **fuera del repo**, copiar `.env.example` a `.env`, rellenarlo y ejecutar `.venv\Scripts\python -m common.db`. Debe decir "Conectado a … como MALACK". El wallet y el `.env` no se suben al repo.
7. **Borrar solo las tablas de prueba** (último bloque del SQL: `DROP TABLE … PURGE`). **La base de datos y el usuario se quedan** para el jueves.

"My recently viewed" del inicio no tiene opción para quitar elementos. Se va renovando con lo que abráis y la entrada rota de `SeguridadOracle` desaparecerá sola. Es inofensiva.

5.2 Resto de tareas de hoy

☐ **Los 4** comprueban que pueden entrar en su cuenta OCI con el doble factor (cuentas de respaldo).
☐ Javier: mirar la fecha de fin del trial (aviso del inicio o *Billing → Subscriptions*).
☐ Ver el vídeo del taller de Oracle del 10/10 (enlace en el apartado 18).
☐ Revisar el Campus Virtual por si hay novedades (aulas, materiales, avisos). El reto no se sabrá hasta el jueves por la mañana.
☐ Decidir las parejas del 14/10: quién hace el kit de NICS y quién el de Mercedes (apartado 6.1).
☐ Acordar los roles del equipo (apartado 7) y quién presenta.
☑ Python en el portátil de Javier: 3.13.7 (lanzador `py`) con `.venv` y todas las dependencias instaladas.
☐ Los otros 3: clonar el repo y seguir el `README` (`py -m venv .venv`, instalar `requirements.txt` y los de los kits, `python -m pytest`). Python 3.11–3.13 vale.
☑ Repositorio Git compartido: github.com/javirmmn/malackathon-2026, privado.
☑ Invitar a los otros 3 al repo: ya son colaboradores (14/10).
☐ Crear un grupo de Telegram o WhatsApp del equipo. Compartir allí la URL del esquema y, por un canal seguro, las credenciales de `MALACK` y el wallet.

Las tareas de 5.2 que siguen sin marcar no están confirmadas; hacedlas hoy si siguen pendientes.

6. Hoy, 14/10: kits de arranque para NICS Lab y Mercedes-Benz

El reto asignado no se sabe hasta el jueves 15 por la mañana. Por eso el miércoles se preparan **kits de arranque** para nuestras dos primeras opciones: código base reutilizable que no depende de los datos del reto (que se dan el mismo día). El jueves, en cuanto se anuncie el reto, se elige el kit que toca y se empieza desde ahí y no desde cero. Si nos tocara NTT, Indra, Google o Betsson, se usan las partes comunes y la preparación ligera del apartado 6.4.

Ojo con las normas: las bases no prohíben llevar herramientas preparadas, pero la solución se evalúa sobre lo que se haga durante el evento con los datos del reto. Preparad **infraestructura genérica** (esqueleto, utilidades, plantillas), no una solución "enlatada". Si hay dudas, preguntad a la organización (empresas.informatica@uma.es) y, si os preguntan, decid con naturalidad qué traíais preparado.

Estado del 14/10 por la tarde: los dos kits están hechos, probados y documentados.

- **Kit NICS** (`kits/nics/`): `python -m kits.nics.run --testbed` clasifica bien **3 de 3** servicios en unos 20 s y genera el informe "qué delató a cada impostor" y el dashboard. Por seguridad, **solo escanea localhost** salvo que se le pase el rango autorizado con `--scope` .
- **Kit Mercedes** (`kits/mercedes/`): con un dataset sintético de 2.000 clientes y 5 segmentos plantados, un comando genera la EDA, la comparación de modelos, los perfiles y el dashboard con simulador en unos 30 s. Recupera los segmentos con un acuerdo ARI de **0,96**.
- **Oracle integrado:** `python -m common.load fichero --tabla T` sube cualquier fichero del reto a Oracle (alternativa a Data Load) y los kits guardan sus resultados con `--a-oracle PREFIJO` . Pendiente de probar contra la base de datos real cuando esté el wallet.
- **Presentación:** plantillas de 7 diapositivas (NICS, Mercedes y genérica), guion y preguntas probables en `docs/presentacion/` ; diagramas en `docs/arquitectura/` .
- **12 tests** en verde (`python -m pytest`). Cómo usar cada kit el jueves: en su `README.md` .
- **Lo que queda para las parejas:** clonar, ejecutar los ensayos, **entender el código** (el jueves hay que explicarlo al jurado) y mejorar lo que vean. Además, la plantilla del PDF y la puesta en común (6.5).

6.1 Organización del día y estructura del repo

Quién	Tarea del 14/10
Pareja A (2 personas)	Kit **NICS Lab** (apartado 6.2)
Pareja B (2 personas)	Kit **Mercedes-Benz** (apartado 6.3)
Todos, al final del día (1 h)	Partes comunes: ~~conexión a Oracle desde Python, plantilla Streamlit~~ (**ya hechas el 13/10** en `common/` ; solo probar la conexión real con el wallet), plantilla del PDF de presentación y puesta en común (cada pareja enseña su kit a la otra)

Para empezar los kits: el dashboard de cada uno parte de una copia de `common/streamlit_app.py` ; para leer y limpiar datos, `common/dataio.py` (`read_any` , `normalize_columns` , `to_number_es` , `quick_profile`); para Oracle, `common/db.py` . Cada kit tiene ya su `README.md` y `requirements.txt` (dependencias instaladas en el portátil de Javier). **Ojo con pandas 3:** las columnas de texto ya no son de tipo `object` ; usad `pd.api.types.is_numeric_dtype` y similares.

Estructura del repositorio (**ya creada** en `C:\Users\jrmro\Malackathon_2026` y en GitHub):

Carpeta	Contenido
`common/` hecho	Conexión a Oracle (`oracledb` + wallet, por variables de entorno), utilidades de carga y exportación de CSV/Excel, plantilla de app Streamlit, estilos de gráficos
`kits/nics/` hecho	Kit de clasificación de servicios (6.2)
`kits/mercedes/` hecho	Kit de clustering de clientes (6.3)
`docs/`	Plan, `oracle.md` (URL del esquema, sin contraseñas), `sql/` con consultas, plantilla del PDF de presentación, diagrama de arquitectura
`data/`	Datos locales; en `.gitignore` . El wallet y el `.env` tampoco se suben
`tests/`	Pruebas de `common/` y de los dos kits, sin Oracle (`python -m pytest` , 9 tests)

6.2 Kit NICS Lab (pareja A) `hecho el 14/10`

Objetivo del kit: que el jueves baste con meter la lista de servicios del entorno del reto para tener las primeras clasificaciones en menos de una hora.

1. **Inventario:** módulo que lee la lista de servicios (dirección, puerto, ruta) desde un CSV o YAML y la normaliza.
2. **Recolector de evidencias:** envía a cada servicio un conjunto configurable de peticiones inocuas, repetidas varias veces, y guarda respuesta, cabeceras, código de estado y latencia (en SQLite o CSV, opcionalmente en Oracle).
3. **Extracción de rasgos por servicio:** variabilidad de respuestas ante la misma petición (similitud textual), estadísticas de latencia (media, desviación), coherencia de cabeceras y códigos de estado, y coincidencias con valores por defecto conocidos de honeypots.
4. **Clasificador por reglas con pesos** (configurables en un YAML): puntuación por clase (real / honeypot clásico / honeypot generativo), clase final, confianza y lista de evidencias que la justifican.
5. **Salidas:** dashboard en Streamlit (tabla filtrable y detalle por servicio) y generador de informe Markdown/PDF con "qué delató a cada impostor".
6. **Banco de pruebas local** para comprobar que todo funciona sin el entorno real: tres servicios simulados en `localhost`: uno "real" (respuestas estáticas coherentes), uno "honeypot clásico" (respuestas fijas por defecto) y uno "generativo" (respuestas generadas por un modelo local con Ollama). El clasificador debe separarlos.

Hecho cuando: `python -m kits.nics.run --targets targets.csv` recorre el banco de pruebas, clasifica bien los 3 servicios y el dashboard los muestra con sus evidencias. ✅ **Cumplido** con `python -m kits.nics.run --testbed`: 3/3. **Ensayado también con una IA real** (Ollama + `llama3.2:3b`, instalado el 14/10): 3/3 tras ajustar el timeout (60 s), la regla de latencia y añadir los timeouts como evidencia. Sin Ollama, el banco de pruebas usa un simulador.

Dependencias: `httpx`, `pandas`, `pyyaml`, `rapidfuzz`, `streamlit`, `fastapi` + `uvicorn` (servicios simulados), Ollama con un modelo pequeño.

6.3 Kit Mercedes-Benz (pareja B) `hecho el 14/10`

Objetivo del kit: que el jueves, con el dataset de clientes, en unas 2 horas haya segmentos perfilados y un dashboard funcionando.

1. **Dataset de ensayo público** de clientes (por ejemplo, *Customer Personality Analysis* o *Mall Customers* de Kaggle), solo para probar el pipeline.
2. **EDA automática:** tipos de variables, nulos, distribuciones, valores atípicos y correlaciones, en un informe HTML que se genera con un comando.
3. **Preprocesado configurable:** detección de numéricas y categóricas, imputación, encoding, escalado (Standard o Robust), tratamiento de valores atípicos y variables tipo RFM si hay fechas o importes.
4. **Búsqueda de modelos:** K-Means (k de 2 a 10), GMM, HDBSCAN y K-Prototypes, con tabla comparativa de silueta, Davies-Bouldin y Calinski-Harabasz, y gráfico del codo.
5. **Perfilado:** tabla de medias y medianas por cluster, variables que más distinguen a cada segmento, gráfico radar y una plantilla para poner nombre de "persona" y acciones de marketing a cada segmento.
6. **Visualización:** PCA y UMAP en 2D coloreados por cluster.
7. **Dashboard Streamlit:** selector de segmento, perfil y simulador de "¿a qué segmento pertenece este cliente?".
8. **Opcional:** carga y lectura desde la Autonomous DB `malackathon` mediante `common/`.

Hecho cuando: con el dataset de ensayo, un solo comando genera la EDA, la comparación de modelos y los perfiles, y el dashboard muestra los segmentos. ✅ **Cumplido.** En lugar de Kaggle (pide registro) se usa un dataset **sintético** con segmentos plantados, lo que permite medir si el pipeline acierta. HDBSCAN se usa desde scikit-learn, sin el paquete `hdbscan`.

Dependencias: `pandas`, `scikit-learn`, `hdbscan`, `umap-learn`, `kmodes`, `plotly`, `streamlit`, `ydata-profiling` (opcional).

6.4 Preparación ligera para el resto de retos (si sobra tiempo o por la noche)

☐ **NTT DATA:** ~~instalar Ollama y descargar un modelo de 3 a 8B~~ (**hecho** en el portátil de Javier: `llama3.2:3b`, unos 4 s por respuesta corta); falta dejar en caché un modelo NLI multilingüe (`transformers`).

☐ **Indra:** instalar `rasterio`, `opencv-python`, `scikit-image` y `folium`; opcionalmente, probar umbrales de color sobre una imagen SWIR pública de Copernicus Browser.

☐ **Google:** la conexión a Oracle de `common/` ya cubre la base del reto.

☐ **Betsson:** `pip install playwright` y `playwright install chromium`.

6.5 Cierre del día 14/10

☑ Ambos kits en `main`, con su `README` (falta hacer commit y push).

☐ Los 4 portátiles clonan el repo, pasan `python -m pytest` y ejecutan los dos kits sin errores.

☐ Comprobar que la base de datos `malackathon` sigue en estado *Available*; si no, *Start*.

☐ Si da tiempo: wallet descargado, `.env` rellenado y `.venv\Scripts\python -m common.db` conectando como `MALACK`. Credenciales y wallet repartidos al equipo por un canal seguro.

☑ Plantillas de la presentación (7 diapositivas: NICS, Mercedes y genérica) en `docs/presentacion/`, con guion y preguntas probables, y diagramas de arquitectura en `docs/arquitectura/`. PDF con `python -m common.pdf docs\presentacion\nics.html`.

☐ Material físico listo (apartado 16) y a dormir pronto: el día 1 dura casi 11 horas.

7. Organización del equipo

Rol	Responsabilidades
Líder / presentador	Controla el tiempo y los checkpoints, habla con los mentores, recoge feedback, prepara el PDF y el guion de la demo, hace la entrega en el Campus Virtual y presenta el viernes (una sola persona).
Datos / backend	Carga y limpieza de datos, Oracle (base de datos, usuario, REST), pipeline principal e integración.
Modelo / IA	Núcleo técnico: modelo, reglas, métricas y validación. Lleva el registro de experimentos.
Frontend / visualización	Dashboard o interfaz de la demo (Streamlit o similar), gráficos, capturas y **vídeo de respaldo** de la demo.

Reglas internas: una reunión de 5 minutos cada 90 minutos para ver qué hay hecho, qué bloquea y qué viene. Ramas cortas y `main` siempre funcionando. Ninguna decisión técnica se discute más de 10 minutos: decide el líder.

8. Plan horario del jueves 15

Franja	Objetivo	Hito de control
09:00 – 10:15	**Se conoce el reto asignado.** Si es NICS o Mercedes, se parte del kit correspondiente; si es otro, de `common/` y la preparación ligera. Registro, presentación general y del reto. **Apuntad los criterios de evaluación**, el formato de entrega y las métricas. Preguntad qué valoran más.	Criterios anotados
10:15 – 11:30	Explorar datos y entorno, adaptar el kit (o montar el esqueleto si no hay kit) y repartir tareas. Primera versión mínima funcionando cuanto antes.	Algo funcionando de principio a fin
11:30 – 15:00	Solución base completa, aunque sea simple. Comer por turnos.	Primera versión enseñada a un mentor
15:00 – 18:00	Mejoras, métricas, el elemento diferenciador y la interfaz de la demo.	Versión 2 con métricas
18:00 – 19:15	**Congelar el código.** PDF, guion, diagrama de arquitectura y vídeo de respaldo de la demo.	Demo ensayada una vez
19:15 – 19:45	**Entrega por el Campus Virtual** (código, PDF, URL del esquema de Oracle si aplica).	Entrega confirmada
21:45	Lista de finalistas. Si estamos, ensayar la presentación esa noche o a primera hora.	—

Clave: los mentores evalúan **por fases durante el día**. Enseñadles avances parciales pronto y a menudo; no esperéis a tenerlo todo.

9. Presentación y defensa (viernes 16)

Plantillas listas en `docs/presentacion/` (`nics.html`, `mercedes.html`, `generica.html`): se rellena lo resaltado en amarillo y se exporta con `.venv\Scripts\python -m common.pdf docs\presentacion\<fichero>.html`. El `README.md` de esa carpeta tiene el guion y las respuestas a las preguntas probables.

ESTRUCTURA DE 7 MINUTOS

Tiempo	Contenido
0:00 – 0:30	El problema en una frase y por qué importa a la empresa.
0:30 – 1:30	Enfoque y arquitectura: un diagrama.
1:30 – 5:30	**Demo en directo**, siguiendo un caso concreto de principio a fin.
5:30 – 6:30	Resultados: métricas, hallazgos y valor de negocio.
6:30 – 7:00	Limitaciones honestas y siguientes pasos.

- Ensayad con cronómetro: 6:45 como máximo.
- Llevad el vídeo de la demo por si falla la red o el directo. El PDF también en un USB.
- Preparad respuestas a preguntas probables: ¿por qué este método?, ¿cómo escala?, ¿qué haríais con más tiempo?, ¿cómo lo habéis validado?

- En el debate puede hablar cualquiera: cada miembro debe saber explicar su parte.

10. Plan por reto · 1 NICS Lab: encuentra a la IA impostora

Ciberseguridad · IA · Honeypots · Reconocimiento

OBJETIVO

Clasificar cada servicio web de la red proporcionada como **real**, **honeypot clásico** u **honeypot generativo**, y documentar las evidencias que lo delatan.

ALCANCE

Trabajar **solo sobre la red y los servicios del entorno del reto** y respetar las normas que den los organizadores.

ENFOQUE

1. **Inventario** de los servicios del entorno autorizado: dirección, puerto, tipo de servicio y primera respuesta.
2. **Batería de pruebas automatizada** que recoja evidencias por servicio:
   - **Consistencia:** ¿la misma petición repetida da siempre la misma respuesta? Un modelo generativo suele variar.
   - **Latencia:** tiempos de respuesta irregulares o altos, típicos de la generación de texto.
   - **Coherencia con el protocolo:** cabeceras, códigos de estado y formatos que cumplan lo esperado de un servicio real.
   - **Valores por defecto** conocidos de software de honeypot clásico.
   - **Coherencia del contenido:** contradicciones entre respuestas, datos inventados o demasiado "verosímiles".
3. **Clasificador por reglas con pesos**: puntuación por categoría, nivel de confianza y lista de evidencias.
4. **Informe y dashboard**: tabla con servicio, clasificación, confianza, evidencias y la técnica que lo delató.

DIFERENCIADOR

Documentación clara y reproducible de cada clasificación. El reto lo pide explícitamente y pesa mucho en la evaluación.

PREPARACIÓN

- Python con `requests` / `httpx`, `pandas` y `streamlit`. Plantilla del informe hecha.
- Repasar cómo funcionan los honeypots (de baja y alta interacción) y por qué los LLM son poco deterministas.

11. Plan por reto · 2 Mercedes-Benz: Clustering the Star

Machine Learning · Análisis de datos · Clustering

PIPELINE

1. **EDA** (análisis exploratorio): tipos de variables, nulos, distribuciones, valores atípicos y correlaciones.
2. **Preprocesado:** imputación, one-hot o target encoding de las categóricas, escalado (Standard o Robust) y, si hace falta, ingeniería de variables tipo RFM (recencia, frecuencia, valor).

3. **Modelos:** K-Means como base; GMM; HDBSCAN o DBSCAN para formas irregulares; K-Prototypes si hay muchas categóricas.
4. **Elegir k:** método del codo, coeficiente de silueta, Davies-Bouldin, y sobre todo que los grupos se puedan interpretar.
5. **Visualización:** PCA o UMAP en 2D coloreado por cluster, y perfiles por cluster (medias o medianas y gráficos radar).
6. **Negocio:** nombre de "persona" para cada segmento, descripción, tamaño, valor y **acciones de marketing concretas** por segmento.

DIFERENCIADOR

Gana el **storytelling de negocio**, no el algoritmo. Al ser el reto de menor dificultad (3), la competencia se decide en los insights y la presentación.

DEMO

Dashboard en Streamlit: filtro por cluster, perfil del segmento y simulador que diga a qué segmento pertenece un cliente nuevo. Si se usa Oracle, datos cargados en la Autonomous DB.

PREPARACIÓN

`pandas`, `scikit-learn`, `hdbscan`, `umap-learn`, `plotly`, `streamlit`, `kmodes`.

12. Plan por reto · 3 NTT DATA: El Guardián

NLP · Análisis de datos · LLM local

ESTRATEGIA

1. **Primera hora:** poner en marcha el **bot de ejemplo** y hacer una primera entrega al ranking.
2. **Comprobaciones baratas:** que las cifras, fechas, nombres propios y entidades de la respuesta aparezcan en las fuentes (normalizando formatos).
3. **Verificación por afirmaciones:** dividir la respuesta en afirmaciones atómicas y comprobar cada una contra las fuentes con un modelo NLI multilingüe (por ejemplo, variantes XNLI de mDeBERTa).
4. **LLM local como juez** (Ollama), con un prompt estricto: "¿está respaldada por las fuentes? sí / no / parcial".
5. **Combinar las señales** en una puntuación y **ajustar el umbral** sobre un conjunto de validación para la métrica del ranking.

PRUEBA SORPRESA A MITAD DEL EVENTO

No sobreajustéis al dataset: reglas generales, umbral robusto y un pipeline fácil de reejecutar con datos nuevos.

PREPARACIÓN (IMPRESCINDIBLE ANTES DEL JUEVES)

- Instalar **Ollama** y **descargar los modelos antes**, porque la red del evento puede ir lenta. Mejor un modelo pequeño y rápido (3–8B) que uno grande.
- Dejar en caché los modelos NLI de Hugging Face. `transformers`, `torch`, `spacy` con modelo en español.

13. Plan por reto · 4 Indra: incendio con Sentinel-2

Teledetección | Observación de la Tierra | Visión por computador

DATOS

Dos composiciones de la misma escena: **True Color** (RGB visible) y **SWIR**. En las composiciones SWIR el fuego activo sale en rojo o naranja muy brillante, la zona quemada en tonos oscuros o pardos y la vegetación sana en verde.

PIPELINE

1. Leer las imágenes con `rasterio` o OpenCV. Comprobar si traen georreferencia (GeoTIFF) o son imágenes planas.
2. **Segmentar** por umbrales en HSV o Lab, o por clustering de píxeles, en: fuego activo, área quemada, humo (más visible en True Color), vegetación, suelo y agua.
3. Limpiar con operaciones morfológicas (apertura y cierre) y quitar regiones pequeñas.
4. **Medidas:** superficie en hectáreas a partir de la resolución del píxel (10 m o 20 m en SWIR), perímetro, frente activo y dirección probable de propagación (por ejemplo, según la pluma de humo).
5. Exportar los polígonos a **GeoJSON** y mostrarlos en un mapa interactivo (`folium`).

OJO

Al ser composiciones RGB y no bandas en bruto, no se puede calcular NBR ni dNBR directamente. Explicad en la presentación cómo se haría con las bandas B8 y B12. Eso suma puntos.

PREPARACIÓN

`rasterio`, `opencv-python`, `numpy`, `scikit-image`, `shapely`, `geopandas`, `folium`. Repasar Sentinel-2 (bandas y resoluciones) y las composiciones SWIR para incendios.

14. Plan por reto · 5 Google: incidente en el GSEC

Ciberseguridad defensiva | IA | Threat hunting

1. **Repositorio de datos:** cargar los 30 días de telemetría en la Autonomous DB de Oracle y normalizar marcas de tiempo, hosts, usuarios e IPs.
2. **Consola web de análisis** (APEX, ORDS + frontend o Streamlit): línea temporal, búsquedas, filtros y pivotes por entidad.
3. **Detecciones:** reglas en SQL para anomalías (accesos fuera de horario, volúmenes salientes anómalos, nuevas cuentas o privilegios), mapeadas a MITRE ATT&CK.
4. **Reconstruir la intrusión:** cronología con el acceso inicial, el movimiento lateral, la exfiltración y el sabotaje, más la lista de IOCs (indicadores de compromiso).
5. **Contención en vivo:** panel de acciones y playbook de respuesta. Un LLM puede resumir los hallazgos.

15. Plan por reto · 6 Betsson: lobby, lanzamiento y jugada

Automatización web | Testing | IA

1. **Playwright** (Python): abrir el lobby, buscar el juego nuevo, verificar que se ve, lanzarlo y esperar a que cargue.
2. Jugar una partida: localizar los controles (a menudo dentro de un canvas o iframe) y hacer una jugada.
3. Guardar **capturas** de cada paso y emitir un **veredicto** (OK / FALLO y motivo) en un informe HTML.
4. Opcional: un modelo con visión que valide las capturas ("¿se ve el juego cargado?").
5. Respetar las condiciones de uso de las webs y el alcance que marque la empresa.

16. Material e instalación previa

COMÚN A TODOS LOS PORTÁTILES

☐ Python 3.11–3.13, `venv`, Jupyter, `pandas`, `numpy`, `scikit-learn`, `matplotlib` / `plotly`, `streamlit` (todo en `requirements.txt` del repo). *Hecho en el portátil de Javier.*

☐ Git configurado y acceso al repo del equipo. VS Code con las extensiones de Python y Jupyter.

☐ Acceso al Campus Virtual y a la consola OCI comprobado (con el doble factor en el móvil).

SEGÚN EL RETO

Reto	Instalar o descargar antes
NICS Lab	`httpx` / `requests`, plantilla del informe
Mercedes-Benz	`hdbscan`, `umap-learn`, `kmodes`
NTT DATA	Ollama con modelos descargados, `transformers`, `torch`, modelo NLI en caché, `spacy` en español
Indra	`rasterio`, `opencv-python`, `scikit-image`, `geopandas`, `folium`
Google	Cliente de Oracle (`oracledb`), wallet de la base de datos descargado
Betsson	`playwright` y `playwright install` (navegadores)

MATERIAL FÍSICO

☐ Portátiles cargados, cargadores y **regleta**.

☐ Móvil para compartir datos como red de respaldo.

☐ USB con el PDF y el vídeo de la demo para el viernes.

☐ Agua y algo de comer: la jornada es larga.

17. Riesgos y planes de contingencia

Riesgo	Plan
La tenancy de Javier falla o se queda sin recursos	Usar la cuenta OCI de otro miembro del equipo (los 4 tenemos). Si alguna no sirve, la organización permite pedir cuenta con un alias @uma.es.
La red del evento va lenta	Modelos y dependencias descargados antes; compartir datos desde el móvil.
No llegamos a todo	Prioridad: versión mínima funcionando, luego demo y luego mejoras. Congelar el código a las 18:00 sí o sí.
Falla la demo en directo	Vídeo grabado y capturas en el PDF.

Se olvida la entrega	El líder entrega entre las 19:15 y las 19:45 y confirma que ha quedado registrada.
Bloqueo técnico de más de 20 min	Preguntar a un mentor o ir a la sala Haz Cosas (aula 4.1.1).
Python no funciona en algún portátil	Resolverlo hoy o mañana. En Windows, usar el lanzador `py` o instalar desde python.org.

18. Contactos y enlaces

- **Organización:** empresas.informatica@uma.es · Durante el evento: sala de co-working **Haz Cosas, aula 4.1.1**.
- **Repositorio del equipo (privado):** github.com/javirmmn/malackathon-2026
- **Encuesta de infraestructura Oracle:** informatica.cv.uma.es/mod/questionnaire/view.php?id=32225
- **Encuesta de selección de retos** (ya cerrada): informatica.cv.uma.es/mod/questionnaire/view.php?id=30635
- **Vídeo del taller de Oracle (10/10):** objectstorage.eu-frankfurt-1.oraclecloud.com/.../video1274765269.mp4
- **Oracle REST Data Services, guía del desarrollador:** docs.oracle.com/.../oracle-rest-data-services-developers-guide.pdf
- **Servidor web en una instancia de Compute (vídeo):** youtube.com/watch?v=IyQ9iyn6QOY
- **Claves SSH para instancias Linux en OCI (vídeo):** youtube.com/watch?v=v5fGdgIrdd0

19. Contexto para retomar con Claude

Copiar este bloque al empezar una conversación nueva:

Participo con mi equipo (4 personas) en el III Malackathon de la ETSI Informática de la UMA: día 1 el jueves 15/10/2026 (trabajo de 10:15 a 20:00, entrega por el Campus Virtual, finalistas a las 21:45) y día 2 el viernes 16/10 (demo en directo de 7 minutos por un solo miembro, con PDF de apoyo). Orden de preferencia enviado: 1 NICS Lab, 2 Mercedes-Benz, 3 NTT DATA, 4 Indra, 5 Google, 6 Betsson. El reto asignado se conoce el jueves 15 por la mañana: **[completar cuando se sepa]**. El 14/10 preparamos kits de arranque para NICS Lab y Mercedes-Benz (apartado 6 del plan). Los 4 tenemos cuenta OCI de una asignatura (no hemos rellenado la encuesta de Oracle porque no se puede repetir). Usamos la tenancy de Javier (`javierroman`, Spain Central/Madrid, free trial) como principal; la base de datos antigua `SeguridadOracle` ya no existe (Oracle la eliminó por inactividad). El 14/10 se creó la Autonomous DB Always Free `malackathon` (Display name `malackathon-2026`, Transaction Processing, 26ai, Madrid, mTLS) con el usuario `MALACK` (Graph/OML/REST) para usarla el jueves. El ensayo de carga con Data Load y SQL salió bien, y la URL del esquema está en `docs/oracle.md`. La organización usa Oracle Autonomous DB (ATP Always Free, usuario con roles Gráfico/OML/REST, carga de Excel con Data Load y entrega de la URL del esquema). Carpeta del proyecto: `C:\Users\jrmro\Malackathon_2026`, que es también el repo privado `github.com/javirmmn/malackathon-2026`. Ya existen `common/` (Oracle por `.env`, lectura de CSV/Excel, gráficos, plantilla Streamlit), `docs/oracle.md`, `docs/sql/prueba_carga.sql` y un `.venv` con Python 3.13. Los dos kits están hechos y probados el 14/10: `kits/nics` (`python -m kits.nics.run --testbed`, 3/3) y `kits/mercedes` (`python -m kits.mercedes.run --data ...`, ARI 0,96 en el ensayo), cada uno con su README de uso para el jueves. El plan completo está en `docs/plan_fuente.html` y `Plan_Malackathon_2026.pdf`.

Fuentes: Bases III Malackathon ETSII UMA; Programa y Normas Malackathon; guía "Cómo crear la base de datos y un usuario en OCI"; hoja de recursos de Oracle (uma-hackaton v1.0.0); información de retos del Campus Virtual. Las fechas y normas pueden cambiar a criterio de la organización: revisad siempre el Campus Virtual.